FATIMA JINNAH WOMEN UNIVERSITY

DEPARTMENT OF SOFTWARE ENGINEERING

Assignment 01

Git, Gitea, GitHub, Git LFS, and GitHub Pages

Name	Hira Faisal Khan
Registration Number	2024-BSE-07
Course Code	SE-665 (Cloud Computing)
Section	A
Submission Date	10 October 2026
GitHub Username	hirafaisalkhan2007-arch
Gitea Username	hirafaisalkhan
Ubuntu Server IP	192.168.132.129
Ubuntu Prompt	hirafaisalkhan2007-arch@ubuntu:~$

Submission Links

Repository	Link
CC (evidence repository)	https://github.com/hirafaisalkhan2007-arch/CC
GitHub – Assignment01	https://github.com/hirafaisalkhan2007-arch/Assignment01
Gitea – Assignment01	http://192.168.132.129:3000/hirafaisalkhan/Assignment01
Portfolio (GitHub Pages)	https://hirafaisalkhan2007-arch.github.io/

1. Introduction

This assignment builds a complete Git workflow across two platforms. Gitea, a self-hosted Git service, is run in Docker on an Ubuntu server. One local repository is then pushed to both Gitea and GitHub, large files are handled with Git LFS, and a portfolio website is published with GitHub Pages.

Tool	Purpose
Ubuntu Server	Hosts Docker and Gitea; used for all terminal work in Tasks 1–3
Docker + Docker Compose	Runs Gitea and its PostgreSQL database as containers from the instructor's compose.yaml
Gitea	Self-hosted Git service; the first remote (named gitea)
GitHub	Public remote (named github), evidence repository CC, and GitHub Pages hosting
Git LFS	Stores large files outside normal Git history, keeping only small pointer files in the repository
HTML / CSS	Used to build the portfolio website

Getting started: In the existing CC repository, the folder Assignments/Assignment01/screenshots was created to hold the evidence. The terminal prompt was checked to show the GitHub username and hostname ubuntu, and Git and Git LFS were confirmed to be installed.

2. Task 1: Install Gitea and Push a Repository from the Ubuntu Server

Task description: Install and run Gitea on the Ubuntu server using Docker, create a separate local Git repository on the same server, and push it to a public Gitea repository using a personal access token.

Step 1.1: Install and Run Gitea

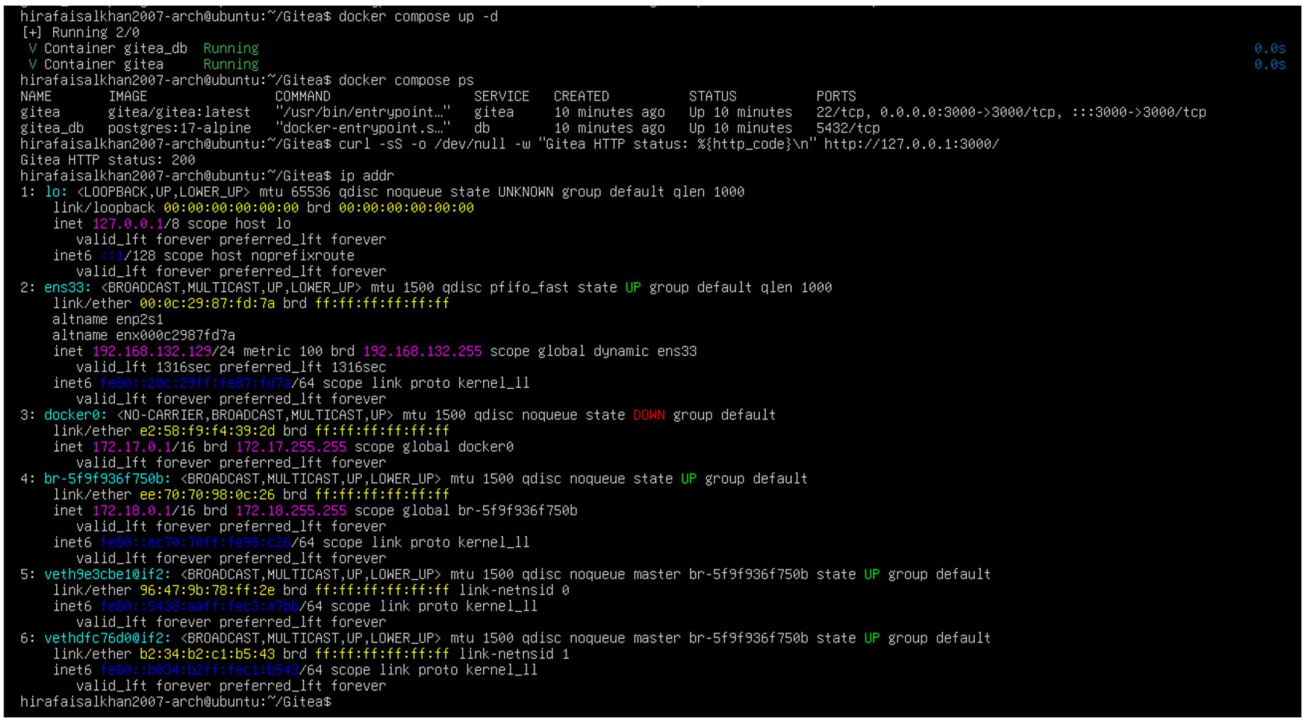
What this step does: Prepares the server and starts Gitea with its PostgreSQL database in Docker containers, then completes the web installation.

- Installed Docker Engine and curl, and added the Docker Compose plug-in.
- Started the Docker service, added my account to the docker group, then signed out and back in so the group change took effect.
- Verified that Git, Docker and Docker Compose all report their versions.
- Cloned the instructor's Gitea setup repository and started Gitea and PostgreSQL with the unchanged compose.yaml.
- Confirmed both containers are running and that Gitea answers on port 3000 with a successful HTTP status.
- Found the server IP (192.168.132.129), opened Gitea in the browser, entered the PostgreSQL database settings, and created the administrator account.

Screenshot: task1_gitea_running.png

Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- The container status check showing both Gitea and PostgreSQL running
- The successful Gitea HTTP status from the local check



task1_gitea_running.png

Step 1.2: Create and Push the Initial Repository

What this step does: Creates the local repository and pushes it to Gitea over the server's local address, authenticating with a token instead of a password.

- Created an empty public repository named Assignment01 in Gitea (no README, .gitignore or license).
- On the server, created a separate local repository on the main branch, outside the Gitea folder and the CC repository.
- Added a README.md with my full name and registration number, and committed it.
- Generated a Gitea personal access token named ubuntu-server-token with repository read/write permission.
- Added a remote named gitea that contains only my username (no token) and checked the remote list.
- Pushed the main branch and pasted the token at the password prompt, so the token never appears in the URL, shell history or screenshot.

Screenshot: task1_gitea_push.png

Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- The push command and the gitea remote with no token in the URL
- A successful push result

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git remote add gitea http://hirafaisalkhan@127.0.0.1:3000/hirafaisalkhan/Assignment01.git
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git remote -v
gitea   http://hirafaisalkhan@127.0.0.1:3000/hirafaisalkhan/Assignment01.git (fetch)
gitea   http://hirafaisalkhan@127.0.0.1:3000/hirafaisalkhan/Assignment01.git (push)
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://hirafaisalkhan@127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 283 bytes | 70.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/hirafaisalkhan/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
```

task1_gitea_push.png

Step 1.3: Verify the Gitea Repository

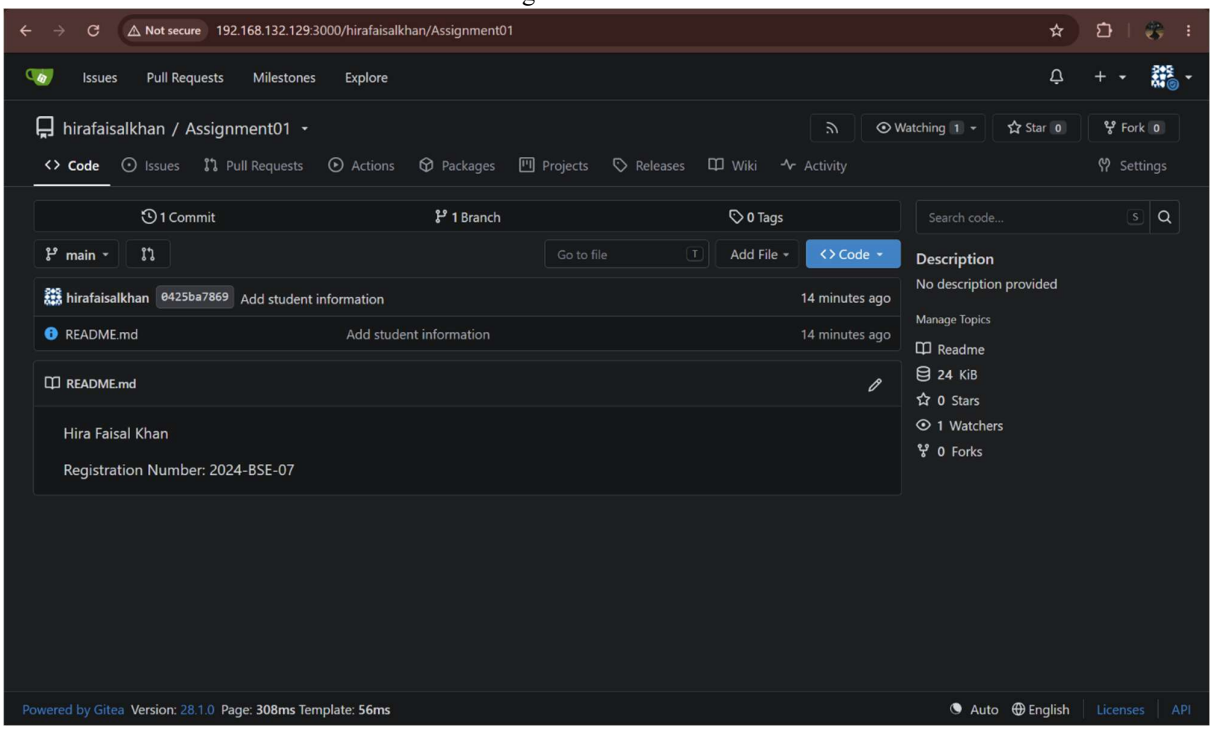
What this step does: Confirms in the browser that the push reached Gitea correctly.

- Opened the Assignment01 repository in Gitea.
- Checked that the owner and repository name are visible, the repository is public, and the README shows my name and registration number.

Screenshot: task1_gitea_repository.png

Shows:

- The Gitea URL in the address bar
- Owner, repository name and Public status
- Rendered README with full name and registration number



task1_gitea_repository.png

Result: Gitea was installed and run on the Ubuntu server, and a separate local repository was pushed to it over HTTP using the local loopback interface.

3. Task 2: Push the Same Repository to GitHub

Task description: A Git repository can have several remotes. Here the same local repository is connected to GitHub as a second remote and pushed there, so one set of commits lives on both Gitea and GitHub.

Step 2.1: Create the GitHub Repository and Verify Both Remotes

What this step does: Prepares GitHub as the second remote and proves both remotes are configured safely.

- Created an empty public GitHub repository named Assignment01.
- In the same local repository, added a remote named github pointing to it.
- Listed the remotes to confirm both gitea and github appear with fetch and push URLs and no credentials.

Screenshot: task2_remotes.png

Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- Both gitea and github remotes with fetch and push URLs
- No token or password in any URL

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ cd ~/Assignment01
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git remote add github https://github.com/hirafaisalkhan2007-arch/Assignment01.git
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git remote -v
gitea   http://hirafaisalkhan@127.0.0.1:3000/hirafaisalkhan/Assignment01.git (fetch)
gitea   http://hirafaisalkhan@127.0.0.1:3000/hirafaisalkhan/Assignment01.git (push)
github  https://github.com/hirafaisalkhan2007-arch/Assignment01.git (fetch)
github  https://github.com/hirafaisalkhan2007-arch/Assignment01.git (push)
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ _
```

task2_remotes.png

Step 2.2: Push to GitHub

What this step does: Uploads the main branch to GitHub, authenticating with a GitHub personal access token entered only at the prompt.

- Pushed the main branch to the github remote and set it as the upstream.
- Confirmed the push finished successfully.

Screenshot: task2_github_push.png

Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- The push to the github remote
- A successful result

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git push -u github main
Username for 'https://github.com': hirafaisalkhan2007-arch
Password for 'https://hirafaisalkhan2007-arch@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 283 bytes | 94.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hirafaisalkhan2007-arch/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'github/main'.
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ _
```

task2_github_push.png

Step 2.3: Verify the GitHub Repository

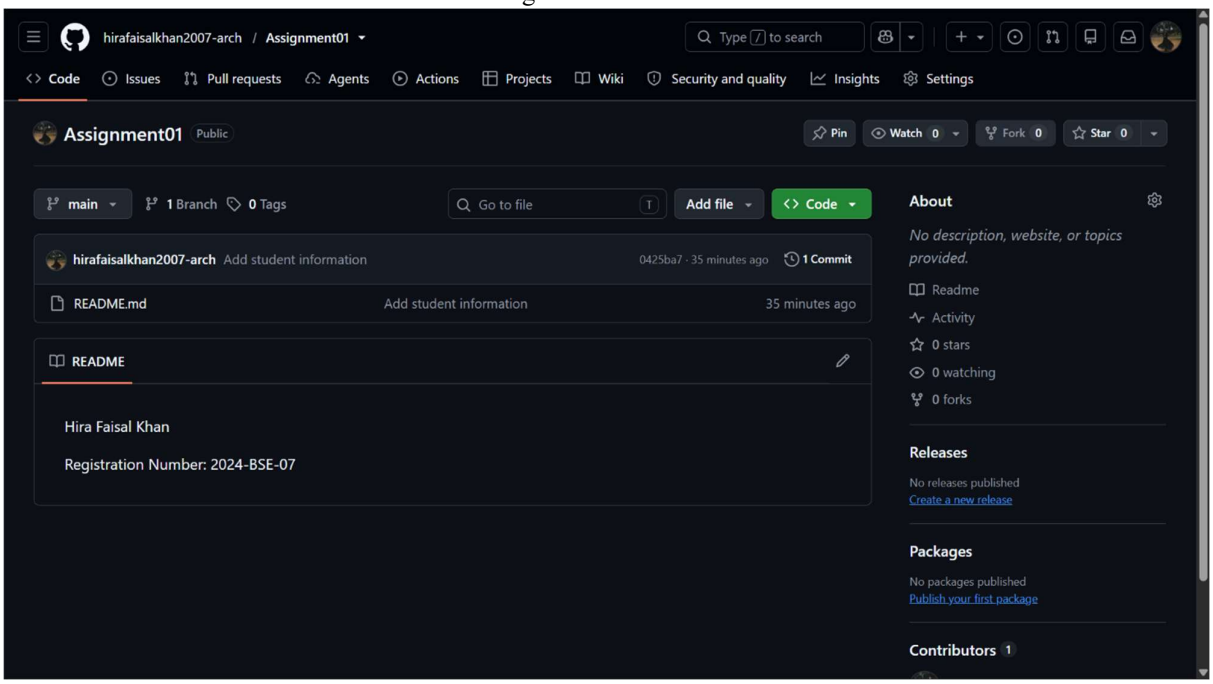
What this step does: Confirms in the browser that the repository is public and shows the correct README.

- Opened github.com/hirafaisalkhan2007-arch/Assignment01.
- Checked the owner, repository name, public status, and the README with my name and registration number.

Screenshot: task2_github_repository.png

Shows:

- Owner and repository name (Assignment01) and Public label
- Rendered README with full name and registration number



task2_github_repository.png

Result: The same local repository is connected to both Gitea and GitHub and is publicly available on GitHub.

4. Task 3: Track Three Large Files with Git LFS

Task description: GitHub rejects ordinary files larger than 100 MiB. Git LFS stores large files on a separate server and keeps only a small pointer file in the repository. In this task, three files larger than 100 MiB are tracked with Git LFS and pushed to GitHub.

Step 3.1: Configure Git LFS

What this step does: Enables Git LFS and tells it to handle all .bin files.

- Initialized Git LFS in the local Assignment01 repository and checked its version.
- Set up tracking for *.bin files.
- Displayed .gitattributes to confirm the *.bin rule was created.

Screenshot: task3_lfs_setup.png

Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- Successful Git LFS initialization and the version
- The tracking command and the *.bin rule in .gitattributes

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ cd ~/Assignment01
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git lfs install
Updated Git hooks.
Git LFS initialized.
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git lfs version
git-lfs/3.7.1 (GitHub; linux amd64; go 1.25.0)
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git lfs track "*.bin"
Tracking "*.bin"
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ cat .gitattributes
*.bin filter=lfs diff=lfs merge=lfs -text
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ _
```

task3_lfs_setup.png

Step 3.2: Create and Verify Three LFS Files

What this step does: Creates three test files, each larger than 100 MiB, and proves Git LFS is managing them.

- Created three different .bin files of about 101 MB, 102 MB and 103 MB.
- Staged .gitattributes together with the three files.
- Listed the LFS-tracked files with their sizes to confirm all three are above 100 MiB.

Screenshot: task3_lfs_files.png

Shows:

- The commands used to create, stage and list the files
- All three files listed with sizes above 100 MiB

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ truncate -s 101M large-file-1.bin
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ truncate -s 102M large-file-2.bin
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ truncate -s 103M large-file-3.bin
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ ls -lh large-file-*.bin
-rw-rw-r-- 1 hirafaisalkhan2007-arch hirafaisalkhan2007-arch 101M Oct 10 13:48 large-file-1.bin
-rw-rw-r-- 1 hirafaisalkhan2007-arch hirafaisalkhan2007-arch 102M Oct 10 13:48 large-file-2.bin
-rw-rw-r-- 1 hirafaisalkhan2007-arch hirafaisalkhan2007-arch 103M Oct 10 13:48 large-file-3.bin
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git lfs ls-files --size
f1753a034b * large-file-1.bin (106 MB)
2338a970c0 * large-file-2.bin (107 MB)
cb91d5908e * large-file-3.bin (108 MB)
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$
```

task3_lfs_files.png

Step 3.3: Commit and Push the LFS Files

What this step does: Commits the files and pushes them to GitHub. The large file contents are uploaded to LFS storage first, then the commit is pushed.

- Committed the three files with the message "Add three files using Git LFS".
- Pushed to the github remote and captured the first successful upload showing the LFS objects completing (100% 3/3).

Screenshot: task3_lfs_push.png
Shows:

- Prompt hirafaisalkhan2007-arch@ubuntu
- LFS upload completing, preferably 100% (3/3)
- A successful push to GitHub

```
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$ git push github main
Username for 'https://github.com': hirafaisalkhan2007-arch
Password for 'https://hirafaisalkhan2007-arch@github.com':
Uploading LFS objects: 100% (3/3), 321 MB | 3.0 MB/s, done.
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 805 bytes | 201.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hirafaisalkhan2007-arch/Assignment01.git
   0425ba7..d09c2dd  main -> main
hirafaisalkhan2007-arch@ubuntu:~/Assignment01$
```

task3_lfs_push.png

Result: Three files larger than 100 MiB are tracked with Git LFS and pushed to the GitHub Assignment01 repository.

5. Task 4: Create a Portfolio or CV with GitHub Pages

Task description: Build a static portfolio with HTML and CSS, store it in a public repository named hirafaisalkhan2007-arch.github.io, and publish it with GitHub Pages.

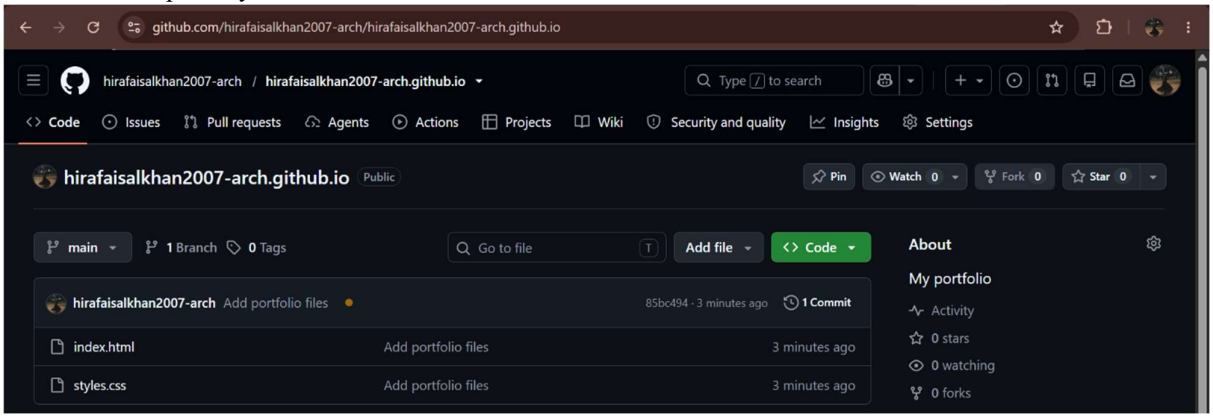
Step 4.1: Create and Push the Portfolio Repository

What this step does: Creates the website source and uploads it to GitHub.

- Created a public repository named hirafaisalkhan2007-arch.github.io.
- Built the portfolio with index.html (structure) and styles.css (design).
- The site contains the four required sections: About Me, Education, Skills and Projects.
- Committed the files and pushed them to the main branch.

Screenshot: task4_pages_repository.png
Shows:

- Repository owner and name
- index.html and styles.css in the file list
- Public repository on the main branch



task4_pages_repository.png

Step 4.2: Publish with GitHub Pages

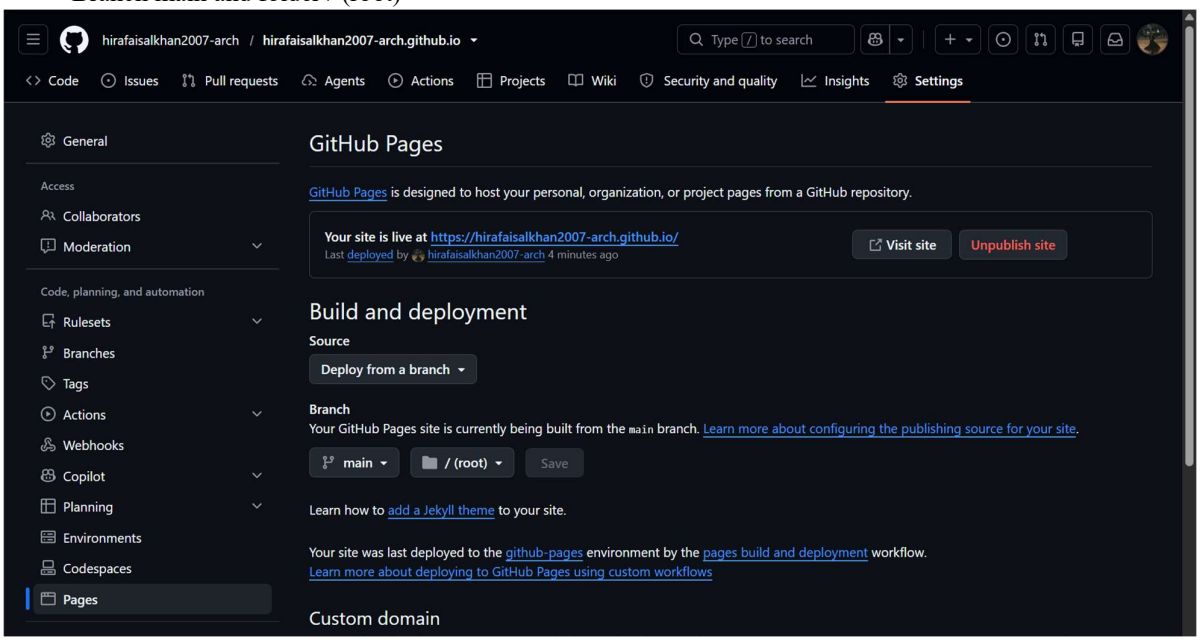
What this step does: Turns on GitHub Pages so GitHub serves the site from the repository.

- Opened Settings → Pages in the portfolio repository.
- Chose Deploy from a branch, selected the main branch and the / (root) folder, and saved.
- Waited until GitHub reported that the site is live.

Screenshot: task4_pages_deployment.png

Shows:

- The GitHub Pages settings page
- Live status and the URL https://hirafaisalkhan2007-arch.github.io/
- Branch main and folder / (root)



task4_pages_deployment.png

Step 4.3: Verify the Live Portfolio

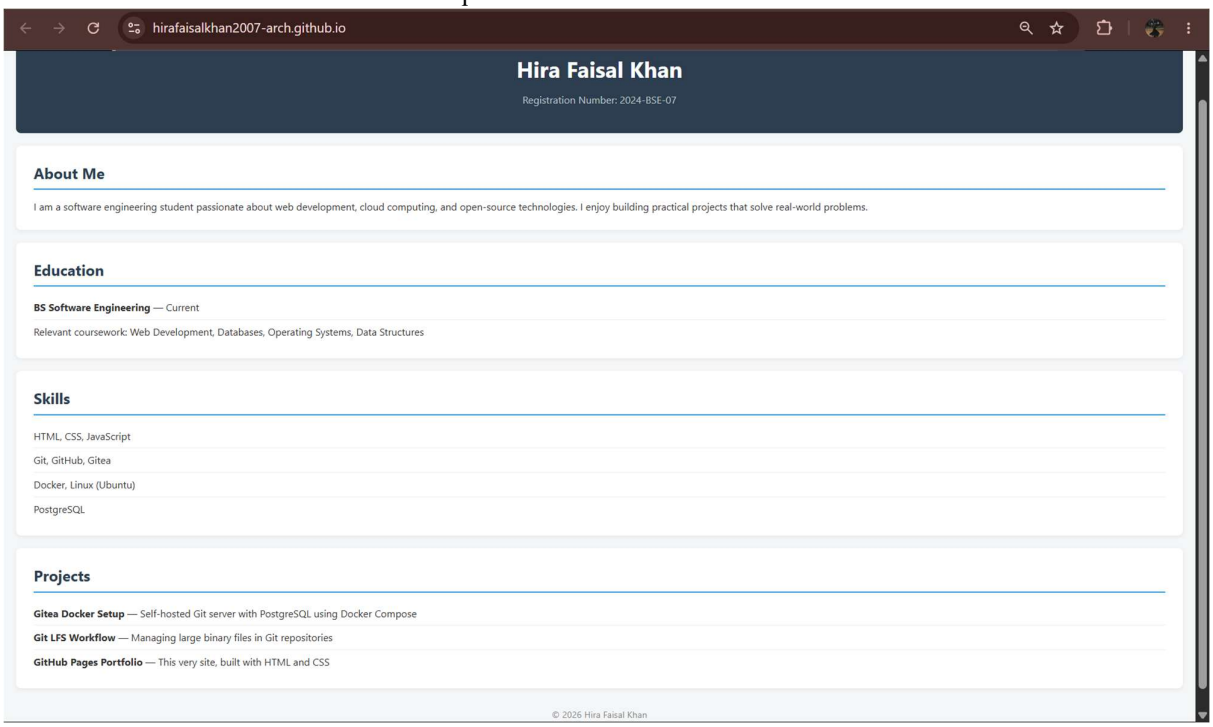
What this step does: Checks that the published website loads correctly in a browser.

- Opened https://hirafaisalkhan2007-arch.github.io/.
- Confirmed the page loads and the required sections are visible.

Screenshot: task4_portfolio_live.png

Shows:

- The full github.io URL in the address bar
- Portfolio content with at least one required section



task4_portfolio_live.png

Result: The portfolio is stored in a public GitHub repository and published as a live GitHub Pages site.